

Received February 11, 2021, accepted February 27, 2021, date of publication March 8, 2021, date of current version March 17, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3064310

Adaptive Traffic Light Control With Quality-of-Service Provisioning for Connected and Automated Vehicles at Isolated Intersections

LEI MIAO¹ AND DALLAS LEITNER²

¹Department of Mechatronics Engineering, Middle Tennessee State University, Murfreesboro, TN 37132, USA

²Department of Computational Science, Middle Tennessee State University, Murfreesboro, TN 37132, USA

Corresponding author: Lei Miao (lei.miao@mtsu.edu)

This work was supported by the Tennessee Department of Transportation and Federal Highway Administration, under Contract RES2021-09.

ABSTRACT Today's traffic intersections primarily use fixed-cycle signaling, which is highly suboptimal and aggravates congestion and waste of fuel in urban areas. In addition, it offers minimal quality-of-service guarantee and makes travel time prediction hard. In this paper, we study how traffic signal scheduling can be done in order to not only reduce average wait time but also provide certain worst-case wait time guarantee to Connected and Automated Vehicles (CAVs). First, we derive the time a CAV spends at an intersection in the best-case and the worst-case, respectively. Using the worst-case wait time result, we propose two novel controllers which assign deadlines to CAVs based on the system state and then perform traffic signal control to meet these deadlines. We use the Simulation of Urban MObility (SUMO) simulator to implement and verify the performance of the proposed controllers. The simulation results show that they not only can guarantee the deadlines, but also outperform other methods such as the default and the delay-based controllers in terms of average delay in all scenarios.

INDEX TERMS Intelligent transportation systems, quality-of-service, real-time systems, traffic signal control.

I. INTRODUCTION

Traffic congestion is a serious issue in urban areas all over the world and has become a severe challenge to the sustainability of economic growth and urbanization. A report from the Texas Transportation Institute says that in 2011, traffic congestion cost drivers \$121 billion in the US alone. Americans also spent 5.5 billion additional hours sitting in traffic in the same year. That came down to \$818 per commuter in lost time and wasted fuel. Traffic congestion, to a large extent, is caused by the inefficiency of today's traffic signal systems, which primarily use fixed-cycle scheduling algorithms. Fixed-cycle approaches are derived off-line based on historical traffic data. The major drawback of fixed-cycle approaches is an open-loop control in the sense that they cannot adapt to traffic fluctuation in real-time and may aggravate the congestion when the traffic pattern changes. Therefore, fixed-cycle traffic signaling is highly sub-optimal and needs to be replaced with better closed-loop solutions.

The associate editor coordinating the review of this manuscript and approving it for publication was Shaohua Wan.

Another problem of today's traffic control system is that it does not provide much quality-of-service (QoS) guarantee. For example, it cannot ensure that each car only spends up to certain amount of time at an intersection. As a result, it is hard to estimate how long it would take a car to travel from point A to point B, especially in case of traffic jam.

Transportation systems are analogous to communication networks in many aspects: cars travel from one place to another whereas packets are sent from one device to another; roads are like communication links where packets are transmitted in a first-come-first-served fashion; and intersections are very similar to routers and switches. A question then naturally arises: *why is QoS readily available in communication networks but very lacking in transportation systems?* The answers to it are multi-fold, but the major reason is that historically, transportation systems simply do not have the necessary information of the cars, such as where and how fast they are and where they are headed.

Fortunately, things have changed dramatically in the past ten years or so: most drivers now have smart phones which are capable of reporting the GPS locations and velocities

of the vehicles; inexpensive and effective electronics (such as induction loops) capable of detecting vehicles are being embedded into the transportation infrastructure; and cars are becoming autonomous with wireless communication capabilities. All the evidence indicates that transportation systems are ready to be smarter and provide QoS provisioning to travelers. Yet, a couple of roadblocks exist and make QoS provisioning harder for transportation applications: First, traffic systems contain both time-driven and event-driven dynamics: cars continuously accelerate and decelerate and have to obey Newton's law; and traffic lights switch between green and red at discrete time instances. Unlike in communication networks where packets are passive objects with no dynamics, the hybrid nature of transportation systems makes it hard to model and control. Second, a communication network is purely technological, but a transportation system involves random and sometimes selfish human actions that are hard to predict.

Connected and Automated Vehicles (CAVs) are expected to reshape the transportation industry in the near future by enhancing the safety and efficiency of the system. Because CAVs do not involve human related decision making and factors such as difference in response time and distracted driving, we believe that it is earlier to provide QoS for CAVs. Therefore, in this initial work towards QoS provisioning for cars, we study how a traffic intersection can improve its efficiency and also provide QoS to CAVs. The contributions of the paper include: (i) Different from existing work in the literature, we incorporate car dynamics into our analysis and derive the best-case and worst-case wait time information for a four-way intersection; (ii) We propose controllers that use the current state of the system to assign a deadline for each CAV and show numerically that the deadlines are always met; and (iii) Our approach works better in terms of average wait time and worst wait time than the fixed-cycle traffic signaling and other existing solutions in most scenarios. Some preliminary results of the paper have been presented in a conference [1]. In this journal version, we expand the analysis to a more generic case that each car has its own driving force and mass; we also improve the proofs and come up with a brand new controller design. In addition, significant simulations are carried out using the well-known Simulation of Urban MObility (SUMO) simulator.

The organization of the rest of the paper is as follows: in Section II, we summarize related work; in Section III, we introduce our system model; in Section IV, we derive the theoretical best and worst wait time a car spends at a 4-way intersection; the controller design and simulation results are presented in Sections V and VI, respectively; finally, we conclude and discuss future work in Section VII.

II. RELATED WORK

In this section, we review related work in traffic signal control.

Perturbation Analysis (PA) and stochastic fluid models [2]–[5] are used to find the performance sensitivity with

respect to certain parameters of traffic light scheduling. Combined with gradient-based algorithms, such approaches are able to iteratively converge to the optimal solution in an on-line setting. Its powerfulness lies in the fact that it does not rely on the stochastic assumptions of the vehicles' arrival and departure processes; in addition, the PA estimator requires very light computation such as counting the number of traffic light switching between certain events.

Petri Nets, a visual and mathematical modeling framework for Discrete Event Systems and hybrid systems, has been widely used in traffic light control [6]–[12]. The main benefit of Petri Nets is that it gives people abilities to not only model but also analyze the system. In particular, the traffic light behavior can be clearly represented using conditions and events, and reachability analysis can be carried out to ensure the proper transitions of traffic lights.

Fuzzy logic controllers are proposed in [13]–[15], in which the theory of fuzzy sets and linguistic control are used to model traffic intersections where analytical methods are in general lacking. Li *et al.* formulate a traffic light control problem that reduces the stop-and-go time and CO_2 emissions [16]. They achieve this goal by using a three-tier structure as well as a branch-and-bound based real-time algorithm.

Ahmad *et al.* [17] considered a scenario that each vehicle has its specific given deadline that needs to be satisfied. The performance of two algorithms: Earliest Deadline First (EDF) and Fixed Priority (FP), in terms of the average delay, is compared with that of fixed-cycle scheduling. In [18], conventional scheduling algorithms such as Minimum Destination Distance First (MDDF) and Minimum Average Destination Distance First (MADDF) are used at intersections to reduce congestion. In addition to scheduling, vehicle routing has also been used in conjunction with traffic light control [19] [20]. Cao *et al.* utilize vehicle agents to help predict road congestion levels, using which a proactive vehicle rerouting strategy based on global distance and local pheromone is employed to assign alternative routes to selected vehicles before they enter congested roads. Combined with traffic light scheduling, their approach is approved to be effective in reducing congestion, air pollution, and fuel consumption. In [20], multicommodity back-pressure algorithms are applied to road networks to control traffic lights and adaptively reroute vehicles and are shown to be able to improve several performance metrics such as queue length, trips completed, travel times, and fair traffic distribution.

Reinforcement Learning (RL) are used in [21] and [22] to adaptively reduce the average cost. A recent work along this line is [23], in which Zhao *et al.* use an RL-based car following model at signaled intersections for CAVs in order to obtain an appropriate driving behaviors to improve a number of things, including road utilization, fuel efficiency, and safety.

[24] and [25] both implement different versions of Q-learning, a type of model-free RL algorithm. In particular, [24] compares the functionality of five different controllers on a small network of four intersections: two fixed-cycle

(long cycle and short cycle) ones, Longest Queue First (LQF), and two versions of Q-learning. The difference between the two Q-learning controllers is that one uses linear regression, while the other uses multiple linear regression, to solve for the cost function. This paper finds that the Q-learning using linear regression has the lowest average number of waiting vehicles, followed by the LQF algorithm, with the short cycle and multiple linear regression Q-learning controllers performing much worse but still outperforming the long cycle fixed-cycle approach. [25] applies a similar method but restricts its simulation to a single four-way intersection: the Q-learning algorithm outperforms fixed-cycle controllers for low-demand traffic, but is worse on varied and higher-demand traffic. In [26], the authors use thresholds of the vehicle waiting queues to adjust the traffic light scheduling; specifically, they use stochastic optimization and Q-learning to dynamically change the thresholds. [27] also applies Q-learning to a multiple intersection model. To do this, the authors use a cooperative deep Q-network with Q-value transfer. By using the Q-values from adjacent intersections as inputs to the model, each agent searches the optimal strategy to control an intersection by a deep Q-network. In order to test the effectiveness of the Q-value transfer, the network is tested on two separate intersection groups. The authors test the network both with and without the Q-value transfer and find that with the transfer in place, there is a drop in the average queue size, the average speed at the intersection increases, and the average waiting time decreases.

Mixed Integer Linear Programming (MILP) is used in [28] to perform adaptive control on an isolated intersection. In [29], MILP is used to find an optimal traffic signal scheduling strategy that strike a good balance between pedestrians' and drivers' needs. Gao *et al.* [30] study a bi-objective traffic light scheduling problem in which the total network-wide delay time of all vehicles and the total delay time of all pedestrians within a finite horizon are minimized. Metaheuristics, including Harmony Search (HS) and Artificial Bee Colony (ABC), are used to solve this problem.

In [31], Khamis *et al.* use Bayesian probability interpretation and the Intelligent Driver Model (IDM) to reduce the average trip waiting time. Protschky *et al.* use the Kalman filter and a generic statistical prediction model to perform traffic light control [32].

A signal model based the double restraint gravitational model is used in [33]. In this paper, the authors use their model to create an origin-destination matrix which is used to attempt to forecast the different traffic flows which lead to the intersection. This method uses a relationship between the traffic attraction and the actual occurrences, with two balancing coefficients, one for each variable. This method is tested against a signal timing based on traditional traffic data, and it leads to a decrease in the average delay for the intersection.

[34]–[36] apply Genetic Algorithms (GA) to optimize intersection traffic flow. Specifically, [34] uses previous

statistical data from an intersection and combines it with various fitness values to optimize the wait time. [35] coordinates the traffic dynamically in simulation using real-world data instead of using a single model. [36] designs and evaluates a GA that dynamically adjusts the fitness model to optimize for either a single intersection or a group of intersections. The testing is broken down into two phases: the first is an intersection that has equal traffic flow in all directions, while phase two has differing levels of traffic for each direction. In both cases, the GA is able to find an optimal time cycle for controlling the traffic lights in order to meet their estimated throughput.

III. SYSTEM MODEL

Although our long-term goal is to perform control on arterials and a network of traffic signals, we restrict ourselves to an isolated four-way intersection in this paper. Fig. 1 shows an illustration of the system model. As indicated earlier, we also assume that the vehicles are CAVs so that the vehicles' dynamics are not affected by human factors such as differences in response time and distracted driving. In case that human-driven vehicles and CAVs co-exist, our results are still applicable when dedicated lanes for CAVs are available. We denote:

L_C : the length of the car. We assume that all the cars have equal length.

L_I : the length of the intersection. We assume that the intersection has equal length in all directions.

L_Q : the maximum length of the vehicle queue. For CAVs, L_Q is not the typical maximum queue length determined by the positions of sensors, e.g., loop detectors. Because CAVs are capable of communicating with the traffic signal controller via V2I wireless communications, this is essentially the maximum distance of reliable wireless transmission.

V_{max} : the speed limit of the roads near the intersection. It is the maximum speed each car can have.

$v(t)$: velocity of a vehicle as a function of time.

d_S : the safe distance between two adjacent cars when they are not in motion.

V_S : the safe velocity. Suppose that two adjacent cars are not in motion and are d_S meters away. When the first one starts to accelerate, V_S is the velocity of the first car at which the second one starts to accelerate. Here, we assume that the two cars are able to learn each other's information, including velocity, via Vehicle-to-Vehicle (V2V) wireless communications or RADAR. Note that $V_S \leq V_{max}$.

N : the maximum number of cars in each queue. For notation ease, we assume that $N = \frac{L_Q}{L_C + d_S}$ is an integer. This can be satisfied by simply choosing L_Q to be the largest integer multiple of $L_C + d_S$ that ensures reliable V2I wireless communications.

T_Y : the duration of yellow light.

D_i^j : the amount of time the i -th car spends in the j -th queue.

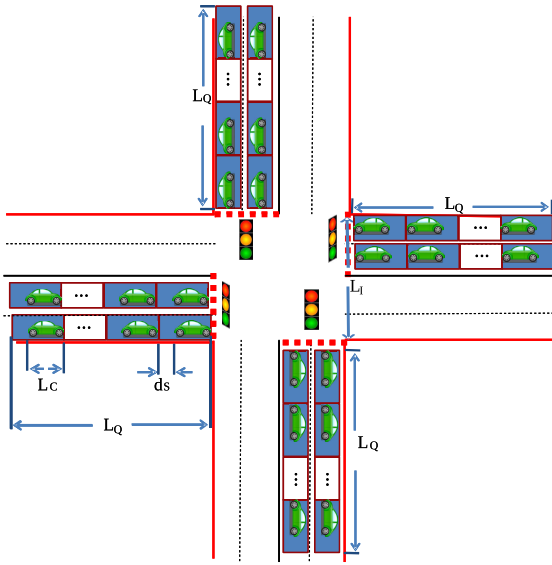


FIGURE 1. System model illustration.

After introducing the system model of the intersection, we now discuss the car model. Since we are interested in providing QoS provisioning to the CAVs using the intersection, we adopt the model used in [37] to estimate the time it takes for the CAVs to travel through the intersection. The road friction $F_1(t)$ and wind dragging force $F_2(t)$ can be calculated as:

$$F_1(t) = c_1 v(t) \quad \text{and} \quad F_2(t) = c_2 v^2(t),$$

respectively, where c_1 and c_2 are constant coefficients. Let a car's mass and driving force be m and F , respectively. Applying Newton's Law, we obtain

$$m\dot{v}(t) + c_1 v(t) + c_2 v^2(t) = F \quad (1)$$

Because the vehicles are subject to V_{max} near an intersection and are typically operated at low speed, we omit the wind's dragging force, which is only large when the velocity is high. (1) can then be reduced to:

$$m\dot{v}(t) + c_1 v(t) = F$$

Taking Laplace transform on both sides and assuming zero initial condition, we get the transfer function, which is first-order:

$$G(s) = \frac{V(s)}{F(s)} = \frac{1}{ms + c_1}$$

The velocity $V(s)$ is:

$$V(s) = \frac{F}{m} \frac{1}{s(s + \frac{c_1}{m})}$$

Let $K = F/c_1$ and $a = c_1/m$, the inverse Laplace Transform yields:

$$v(t) = K(1 - e^{-at}) \quad (2)$$

Because each vehicle may have different mass and driving force, we use K_i^j and a_i^j to denote the two constant coefficients in (2) for the i -th vehicle in the j -th queue. In our later discussion that does not restrict ourselves to a specific queue, we omit the superscript j and use K_i and a_i to denote the coefficients of the i -th vehicle in a queue.

IV. DERIVATION OF BEST AND WORST CASE WAIT TIME

Our goal is to design an adaptive traffic light controller which is capable of providing QoS provisioning. In this section, we first answer the following question: once a car enters a queue, what is the best and worst-case time that it will spend at the intersection? The answer to this question is crucial because choosing the right vehicle deadline in controller design is dependent upon these two values. We use D_{min} and D_{max} to denote them, respectively. D_{min} corresponds to the best case scenario where a car enters a queue and travels through the intersection at velocity V_{max} . Therefore,

$$D_{min} = \frac{L_Q + L_I}{V_{max}}.$$

D_{max} depends on the traffic light control algorithm. We emphasize that we are looking for the maximum delay introduced by the best scheduling algorithm. i.e.,

$$D_{max} = \min_{\text{All traffic controllers}} \max_{\text{All traffic conditions}} \max(D_i^j).$$

Note that $\max(D_i^j)$ is the worst-case delay among all vehicles, and it depends on both the traffic condition and the traffic light controller. It is maximized when the traffic is extremely heavy in both directions so that even if the green light is constant on in one direction, the intersection will never idle. To obtain D_{max} , we first establish the following result.

Lemma 1: Suppose that N cars are sitting in a queue and waiting for the green light. Assume that L_I is wide enough for the first car to accelerate to V_{max} before leaving the intersection. If the green light is turned on and stays on, it takes T_i seconds for the i -th car to leave the intersection, $i \in \{1, \dots, N\}$:

$$T_i = T_{i,w} + T_{i,a} + T_{i,t}, \quad \text{where}$$

$$T_{i,w} = \begin{cases} 0, & \text{if } i = 1 \\ \sum_{k=1}^{i-1} \frac{\ln K_k - \ln(K_k - V_S)}{a_k}, & \text{if } i > 1 \end{cases}$$

is the time for the car in the front (if any),

i.e., the $(i-1)$ th car, to reach the safe velocity V_S ;

$$T_{i,a} = \frac{1}{a_i} [\ln K_i - \ln(K_i - V_{max})]$$

is the time for it to accelerate to V_{max} ; and

$$T_{i,t} = \frac{(i-1)(L_C + d_S) + L_I - (K_i T_{i,a} - \frac{V_{max}}{a_i})}{V_{max}}$$

is the time for it to travel through the rest of the intersection at maximum speed. (3)

Proof: We use induction to prove it.

Step 1: We first focus on T_1 , the time it takes the first car to depart. Because there are no other cars in front of it, $T_{1,w} = 0$ and $T_1 = T_{1,a} + T_{1,t}$. $T_{1,a}$ can be calculated using (2):

$$T_{1,a} = \frac{1}{a_1} [\ln K_1 - \ln(K_1 - V_{\max})] \quad (4)$$

The distance the first car travels during the transient period is:

$$\int_0^t v(\tau) d\tau = K_1 t - \frac{K_1}{a_1} (1 - e^{-a_1 t}) \quad (5)$$

Using (4) and (5), the distance the first car travels from 0 to $T_{1,a}$ is:

$$\int_0^{T_{1,a}} v(\tau) d\tau = K_1 T_{1,a} - \frac{V_{\max}}{a_1} \quad (6)$$

Then, we calculate $T_{1,t}$:

$$T_{1,t} = \frac{L_I - (K_1 T_{1,a} - \frac{V_{\max}}{a_1})}{V_{\max}} \quad (7)$$

Step 2: Assuming that T_{i-1} , $i \in \{2, \dots, N\}$, is known, we now figure out T_i . The difference between T_i and T_{i-1} , ΔT_i , has three parts. The first part is the time the i -th car has to wait for the $(i-1)$ -th car's velocity to reach V_S . Let $\Delta T_{i,w} = T_{i,w} - T_{i-1,w}$, and using (2), we get

$$\Delta T_{i,w} = \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_S)] \quad (8)$$

Therefore,

$$T_{i,w} = \sum_{k=1}^{i-1} \frac{\ln K_k - \ln(K_k - V_S)}{a_k}$$

The second part is the difference between the acceleration time to $V_{\max}$:

$$\begin{aligned} \Delta T_{i,a} &= T_{i,a} - T_{i-1,a} \\ &= \frac{1}{a_i} [\ln K_i - \ln(K_i - V_{\max})] \\ &\quad - \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_{\max})] \end{aligned} \quad (9)$$

Therefore,

$$T_{i,a} = \frac{1}{a_i} [\ln K_i - \ln(K_i - V_{\max})] \quad (10)$$

The third part is $\Delta T_{i,t}$, the difference in time that the cars travel through the intersection at $V_{\max}$. This part essentially is due to the difference in distance that the cars are moving at the maximum speed, and it has two components. The first one is caused by the extra distance $L_C + d_S$:

$$\frac{L_C + d_S}{V_{\max}} \quad (11)$$

and the second one is caused by the difference in distance it takes to reach $V_{\max}$.

$$\frac{(K_{i-1} T_{i-1,a} - \frac{V_{\max}}{a_{i-1}}) - (K_i T_{i,a} - \frac{V_{\max}}{a_i})}{V_{\max}} \quad (12)$$

Combining (11) and (12), we have

$$\begin{aligned} \Delta T_{i,t} &= T_{i,t} - T_{i-1,t} \\ &= \frac{L_C + d_S}{V_{\max}} \\ &\quad + \frac{(K_{i-1} T_{i-1,a} - \frac{V_{\max}}{a_{i-1}}) - (K_i T_{i,a} - \frac{V_{\max}}{a_i})}{V_{\max}} \end{aligned} \quad (13)$$

That is:

$$T_{i,t} = \frac{(i-1)(L_C + d_S) + L_I - (K_i T_{i,a} - \frac{V_{\max}}{a_i})}{V_{\max}} \blacksquare$$

Corollary 1: For any two adjacent cars in a queue that pass the intersection during the same green light period, the difference between their departure time is fixed, regardless of their positions in the queue when the green light is turned on.

Proof: This can be seen by looking at the difference between T_i and T_{i-1} in the proof of Lemma 1. In particular, (8), (9), and (13) only depend on the vehicle specific coefficients a and K , not the positions of vehicles i and $i-1$ in the queue. ■

We now use the results established in Lemma 1 to find $D_{\max}$, which is achieved when the traffic is heavy in both directions. Next, we will not give a precise definition of “heavy”; instead, we give one scenario that will be guaranteed to yield $D_{\max}$. Let us assume that the maximum service rate the intersection provides to the cars is $\mu_{\max}$; this is obtained when cars in one queue are traveling through the intersection at velocity $V_{\max}$ with the right safe distance between adjacent ones. Then, the traffic condition can be considered as heavy or congested when the vehicle arrival rate to each queue is at least $\mu_{\max}$ at all times. This traffic condition will lead to the maximum delay $D_{\max}$ and will be used in later lemmas and their proofs.

Before we continue, we introduce an assumption about the yellow light duration.

Assumption 1: $T_Y > \frac{L_C + d_S}{V_{\max}}$

The right hand side of the inequality above is the time it takes to travel through the car length and the safe distance at the maximum speed, and it is typically a small value. For example, when $L_C = 4.5m$, $d_S = 2m$, and $V_{\max} = 40km/h$, it takes 0.59s to travel through this distance. This assumption ensures that the yellow light duration is not too short and is not causing safety and other issues.

Lemma 2: Assuming that there are N static cars sitting in each queue when the traffic light is turned green for that queue due to heavily congested conditions, the optimal traffic control policy that yields $D_{\max}$ is to let all N cars in each queue pass the intersection during a single green light period.

Proof: Suppose that $M \in \mathbb{Z}^+$ is the number of green light periods that are used to service all N cars. When $M = 1$, the time it takes for the i -th vehicle, $i \in \{1, \dots, N\}$, to leave the intersection is T_i . We use $\tilde{T}_i$ to denote the time it takes for the i -th vehicle to depart when $M > 1$; in this case, let $n_k \in \mathbb{Z}^+$ be the number of cars leaving the intersection during the k -th green cycle, $k \in \{1, \dots, M\}$, and

we have $\sum_{k=1}^M n_k = N$. Next, we use induction to show that $T_i \leq \tilde{T}_i, \forall M$.

Step 1: Regardless of the value of M , the first car leaves the intersection during the first green light cycle, i.e., $T_1 = \tilde{T}_1$. Therefore, $T_1 \leq \tilde{T}_1$ is satisfied.

Step 2: Suppose that $T_{i-1} \leq \tilde{T}_{i-1}, i \in \{2, \dots, N\}$. We discuss two cases to show that $T_i \leq \tilde{T}_i$:

Case 1: Vehicles $i-1$ and i belong to two different batches when $M > 1$, i.e., they pass the intersection during two different green light cycles. In this case, we have

$$\tilde{T}_i = \tilde{T}_{i-1} + T_R + T_Y + \frac{1}{a_i} [\ln K_i - \ln(K_i - V_{\max})] + \frac{L_I - (K_i T_{i,a} - \frac{V_{\max}}{a_i})}{V_{\max}}$$

where T_R is the right light duration after the green cycle during which vehicle $i-1$ leaves the intersection, T_Y is the yellow light duration in the other direction after the right light interval, and the last two terms are the time it takes for the i -th car to pass the intersection in its own green light cycle as the first car in the batch. From Step 2 in the proof of Lemma 1, we have

$$\begin{aligned} T_i &= T_{i-1} + \Delta T_i \\ &= T_{i-1} + \Delta T_{i,w} + \Delta T_{i,a} + \Delta T_{i,t} \\ &= T_{i-1} + \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_S)] \\ &\quad + \frac{1}{a_i} [\ln K_i - \ln(K_i - V_{\max})] \\ &\quad - \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_{\max})] \\ &\quad + \frac{L_C + d_S}{V_{\max}} \\ &\quad + \frac{(K_{i-1} T_{i-1,a} - \frac{V_{\max}}{a_{i-1}}) - (K_i T_{i,a} - \frac{V_{\max}}{a_i})}{V_{\max}} \end{aligned}$$

Therefore,

$$\tilde{T}_i - T_i = \tilde{T}_{i-1} - T_{i-1} \quad (14)$$

$$+ T_R + T_Y - \frac{L_C + d_S}{V_{\max}} \quad (15)$$

$$+ \frac{L_I - (K_{i-1} T_{i-1,a} - \frac{V_{\max}}{a_{i-1}})}{V_{\max}} \quad (16)$$

$$+ \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_{\max})] \quad (17)$$

$$- \frac{1}{a_{i-1}} [\ln K_{i-1} - \ln(K_{i-1} - V_S)] \quad (18)$$

From the induction assumption in Step 2, (14) ≥ 0 ; regardless of the value of T_R , (15) > 0 due to Assumption 1; (16) > 0 because we assume the width of the intersection is greater than the distance needed to accelerate to $V_{\max}$; and finally, (17) + (18) > 0 because it takes longer to accelerate to $V_{\max}$ than V_S . Therefore, we have $T_i \leq \tilde{T}_i$.

Case 2: Vehicles $i-1$ and i belong to the same batch when $M > 1$, i.e., they pass the intersection during the same green

light cycle. Invoking Corollary 1, we have

$$\tilde{T}_i - \tilde{T}_{i-1} = T_i - T_{i-1}$$

From the above equation and the induction assumption in Step 2 that $T_{i-1} \leq \tilde{T}_{i-1}$, we have $T_i \leq \tilde{T}_i$. ■

Note that in addition to traffic condition and the control mechanism, $D_{\max}$ also depends on the two vehicle specific coefficients K and a . In particular, $D_{\max}$ is achieved when all the cars have the smallest driving force $F_{\min}$ and the largest mass $m_{\max}$. In this case, we let $K_{\min}$ and $a_{\min}$ denote the corresponding K and a coefficients, respectively. T_i in Lemma 1 becomes:

$$\begin{aligned} T_i &= T_{i,w} + T_{i,a} + T_{i,t}, \text{ where} \\ T_{i,w} &= \begin{cases} 0, & \text{if } i = 1 \\ (i-1) \frac{\ln K_{\min} - \ln(K_{\min} - V_S)}{a_{\min}}, & \text{if } i > 1 \end{cases} \\ T_{i,a} &= \frac{1}{a_{\min}} [\ln K_{\min} - \ln(K_{\min} - V_{\max})] \\ T_{i,t} &= \frac{(i-1)(L_C + d_S) + L_I - (K_{\min} T_{i,a} - \frac{V_{\max}}{a_{\min}})}{V_{\max}} \quad (19) \end{aligned}$$

ΔT_i in Lemma 1 now becomes ΔT and only contains two parts:

$$\begin{aligned} \Delta T &= \Delta T_w + \Delta T_t, \text{ where} \\ \Delta T_w &= \frac{1}{a_{\min}} [\ln K_{\min} - \ln(K_{\min} - V_S)] \\ \Delta T_t &= T_{i,t} - T_{i-1,t} = \frac{L_C + d_S}{V_{\max}} \end{aligned}$$

Lemma 3: Suppose that (i) In heavily congested conditions, the optimal traffic control policy is used, i.e., N cars in each queue are allowed to pass the intersection during a single green light period and (ii) All the cars have coefficients $K_{\min}$ and $a_{\min}$. For the N cars sitting in the queue and waiting for the green light to be turned on, let $\Delta_{i+1,i}$ be the difference between the total time the $i+1$ -th and the i -th car spend in the queue. Then, $\Delta_{i+1,i} \leq 0$.

Proof: To prove this, we need to investigate when a car moves into a queue and when it leaves an intersection. In particular, we examine the behavior of a car outside of the queue when the green light is turned on for this queue. At that moment, there are N cars sitting in the queue and N cars sitting out of the queue because it is heavily congested. The cars outside of the queue will be entering the queue whenever spaces are available in the queue. Invoking Lemma 2, the N cars in the queue will be serviced and then after the yellow light interval, and N cars in the other queue will be serviced. Therefore, the N cars outside of the queue will not be serviced until $2T_N + 2T_Y$ seconds later. Now, consider the i -th car outside of the queue. It takes $(N + i - 1)\Delta T_w$ seconds for all the cars in front of it to accelerate to V_S . Let $T_{i,q}$ be the time it takes for the i -th car outside of the queue to enter the queue after it starts to accelerate. The total time for the i -th car to enter the queue after the green light is turned on is:

$$(N + i - 1)\Delta T_w + T_{i,q} \quad (20)$$

The amount of time the car spends in the queue before the next green cycle arrives is:

$$2T_N + 2T_Y - (N + i - 1)\Delta T_w - T_{i,q} \quad (21)$$

The total time this car spends in the queue is:

$$2T_N + 2T_Y - (N + i - 1)\Delta T_w - T_{i,q} + T_i \quad (22)$$

The difference of the time the $(i + 1)$ -th and the i -th car spend in the queue is:

$$\begin{aligned} \Delta_{i+1,i} &= -\Delta T_w + T_{i,q} - T_{i+1,q} + T_{i+1} - T_i \\ &= -\Delta T_w + T_{i,q} - T_{i+1,q} + \Delta T_w + \Delta T_t \\ &= -(T_{i+1,q} - T_{i,q}) + \Delta T_t \end{aligned}$$

Because $T_{i+1,q} - T_{i,q}$ is the time it takes to travel distance $L_C + d_S$ at average speed less or equal to V_{max} and ΔT_t is the time it takes to travel the same distance at velocity V_{max} , we get: $\Delta_{i+1,i} \leq 0$. ■

Lemma 3 actually shows something very intuitive: in a queue packed with cars waiting for the traffic light to turn green, the cars closer to the intersection spend not less total time in the queue than those further away from the intersection, i.e., the cars closer to the intersection tend to be the unlucky ones that spend longer time in the queue. They would be luckier if they passed the intersection in the previous green light interval.

Corollary 2: Suppose that all the cars have coefficients K_{min} and a_{min} . $D_{max} = 2T_N + 2T_Y - N\Delta T_w + T_1$.

Proof: D_{max} can be obtained using Lemma 3, i.e., the maximum delay occurs to the first car in the queue waiting for the traffic light to turn green. Specifically, we let $i = 1$ in (22) and get the closed form expression of D_{max} shown above. ■

Since we now have D_{min} and D_{max} , adaptive traffic light controllers can be built. Intuitively, the traffic controller should use the state of the system to intelligently assign the deadline for the cars in each queue, based on which, the traffic lights are toggled between green and red. In what follows, we will present a simple but effective controller, which uses the theoretical results above to perform traffic light scheduling.

V. ADAPTIVE TRAFFIC LIGHT CONTROL WITH QoS PROVISIONING

We introduce the following notations.

Q_{1n} : the queue in the north bound direction.

Q_{1s} : the queue in the south bound direction.

Q_{2w} : the queue in the west bound direction.

Q_{2e} : the queue in the east bound direction.

$N_{1n}(t)$: the number of vehicles in the north bound queue at time t .

$N_{1s}(t)$: the number of vehicles in the south bound queue at time t .

$N_{2w}(t)$: the number of vehicles in the west bound queue at time t .

$N_{2e}(t)$: the number of vehicles in the east bound queue at time t .

The system state at time t , S_t , is a function of the current number of vehicles in the system:

$$S_t = (N_{1n}(t), N_{1s}(t), N_{2w}(t), N_{2e}(t))$$

Let

$$N_1(t) = \max(N_{1n}(t), N_{1s}(t)) \text{ and}$$

$$N_2(t) = \max(N_{2e}(t), N_{2w}(t))$$

We use V_i to denote the i -th vehicle that enters the system, and let a_i be the arrival time of V_i .

Our controller design has two variations, in which the key difference is if the assigned deadline is allowed to change.

A. ADAPTIVE TRAFFIC LIGHT CONTROLLER USING STATIC VEHICLE DEADLINE (ATC-SD)

Upon the arrival of V_i , we assign a deadline d_i to it based on the number of vehicles in the queues:

$$d_i = \begin{cases} a_i + T_{N_2(a_i)} + T_Y + T_{N_{1n}(a_i)} + \alpha, & \text{if } V_i \text{ is in } Q_{1n} \\ a_i + T_{N_2(a_i)} + T_Y + T_{N_{1s}(a_i)} + \alpha, & \text{if } V_i \text{ is in } Q_{1s} \\ a_i + T_{N_1(a_i)} + T_Y + T_{N_{2e}(a_i)} + \alpha, & \text{if } V_i \text{ is in } Q_{2e} \\ a_i + T_{N_1(a_i)} + T_Y + T_{N_{2w}(a_i)} + \alpha, & \text{if } V_i \text{ is in } Q_{2w} \end{cases} \quad (23)$$

where the 2nd through the 5th terms in each case are: the time it takes for all the cars in the opposite direction to leave the intersection, the duration of the yellow light, the time it takes for all cars before V_i in the same queue to leave, and a small constant accounting for disturbances and random factors in the process, respectively. The information that the controller needs is mostly the system state S_t collected from the intersection. Note that the second and fourth terms in each case can be calculated using either (3) if the K and a values of all the vehicles are known or (19) when only K_{min} and a_{min} are known. In case that the vehicle dynamics are totally unknown, these two terms could be estimated by observing the actual system time of the vehicles. We also point that in ATC-SD, d_i is kept unchanged after it is assigned. The deadline reflects the worst-case wait time of the V_i upon its arrival.

ATC-SD works as follows: once d_i is determined for each vehicle in the system, the controller then compares the current estimated departure time for each vehicle in the queue waiting for the green light (details of how it is estimated will be provided later when we present the control algorithm). When a vehicle waiting for the green light is about to miss its deadline, the controller switches the direction of the traffic light.

Some traffic signal controllers enforce a minimum green time before switching in order not to cause the situation that the traffic signals toggle between red and green cycles too fast. Our proposed controller does not suffer from the instability problem and does not have the minimum green time

implementation The reason is that when a car enters a queue, its deadline is determined using a very conservative way: we add the arrival time with the worst-case time it would need for this car to pass the intersection. The latter worst-case time is calculated by assuming that (i) the traffic light becomes red immediately upon the arrival of this car and (ii) all other existing vehicles in the queues will depart the intersection before this car. By doing so, the controller strongly discourage the switching between green and red cycles and only changes the light when it is absolutely necessary. Another way to think about this is as follows: suppose that the traffic light changes to green so as not to miss the deadline of car V_i , then all the cars that arrived before V_i in the conflicting direction will not cause the traffic signal to toggle because they should have already left the intersection before the light turns green for V_i ; those cars that have arrived after V_i in the conflicting direction will not toggle the light immediately either because their deadlines are calculated to allow V_i to leave the intersection first.

B. ADAPTIVE TRAFFIC LIGHT CONTROLLER USING DYNAMIC VEHICLE DEADLINE (ATC-DD)

In a different controller design, we allow the vehicle deadline d_i to be updated when the state of the system changes. The update is similar to (23), but now d_i is a function of time and dependent upon the system state:

$$d_i(t) = \begin{cases} a_i + T_{N_2(t)} + T_Y + T_{N_{1n}}(t) + \alpha, & \text{if } V_i \text{ is in queue } Q_{1n} \\ a_i + T_{N_2(t)} + T_Y + T_{N_{1s}}(t) + \alpha, & \text{if } V_i \text{ is in queue } Q_{1s} \\ a_i + T_{N_1(t)} + T_Y + T_{N_{2e}}(t) + \alpha, & \text{if } V_i \text{ is in queue } Q_{2e} \\ a_i + T_{N_1(t)} + T_Y + T_{N_{2w}}(t) + \alpha, & \text{if } V_i \text{ is in queue } Q_{2w} \end{cases} \quad (24)$$

This slight change gives the system the ability to dynamically adjust the deadlines based on the current system state, and it could be very helpful in some scenarios. For example, some cars may be given tighter deadlines upon arrival when the traffic is light; without the dynamic deadline, the controller would frequently toggle the traffic lights in order to satisfy these cars even when the traffic becomes heavy later on. In this scenario, the dynamic deadline approach would encourage the controller not to change the traffic light states frequently, which is beneficial because the throughput would suffer every time the light changes.

C. ALGORITHM

We now present the pseudo-code of the controller algorithm for both ATC-SD and ATC-DD. For vehicle V_i waiting for the green light in a queue at time t , we let $\tilde{x}_i(t)$ be the estimated departure time of V_i if the traffic light changes at that time instant:

$$\tilde{x}_i = t + T_Y + T_{F_i(t)+1}$$

where $F_i(t)$ is the number of vehicles in front of V_i in the same queue at time t , T_Y is the yellow light duration, and $T_{F_i(t)+1}$ represents the estimated time for V_i to leave the intersection after the green light is on. The control algorithm is shown in Algorithm 1.

Algorithm 1 Adaptive Traffic Light Controller Implementing Both Static and Dynamic Vehicle Deadline Assignment

```

1: while true do
2:   when  $V_i$  arrives, assign  $d_i$  according to equation (23)
3:   //Update Lights
4:   for each car  $V_i$  waiting for green light do
5:     if set to ATC-DD then
6:       update assigned  $d_i$  according to equation (24)
7:     end if
8:      $\tilde{x}_i(t) = t + T_{F_i(t)+1} + T_Y$ 
9:     if  $\tilde{x}_i(t) == d_i - \epsilon$  then // $\epsilon$  is a small positive value
10:      Change light direction
11:      break
12:    end if
13:   end for
14: end while

```

VI. SIMULATION RESULTS

A. SIMULATION PLATFORM AND ENVIRONMENT

To test out the adaptive traffic controllers, we simulate an isolated four-way traffic intersection using SUMO, which is an open source traffic simulation package developed mostly by employees of the Institute of Transportation Systems at the German Aerospace Center. With SUMO, a user is able to design or import various traffic networks. In our simulation, we have created a basic intersection consists of two roads, each supporting bidirectional traffic with two lanes for each direction as shown in Fig. 2. Vehicles are only able to drive in a straight line, and no left/right turns are allowed. For example, if a vehicle enters from the north, it will drive south until it reaches the intersection; once it is able to pass the intersection it will then exit the simulation via the south exit.

Most of the simulation parameters used by SUMO can be specified using user-made XML files. One of these files contains the route data, which include the specifications of every vehicle type, defined routes for vehicles to take, and declarations for every vehicle that will appear in the simulation. The vehicle type creates an object definition for what parameters a specific type of vehicle will have. In our simulations, only one vehicle type was created. The ID of this vehicle type was named "car", which has an acceleration of $6m/s^2$, deceleration of $9m/s^2$, a length of $2m$, and max speed of $100m/s$ (no vehicles in this simulation use their max speed as the street's speed limit is $15m/s$). The routes are predetermined paths that a vehicle can take and are created by specifying which roads a vehicle should take. As previously mentioned, all routes are straight lines through the intersection. Lastly, every individual vehicle in the simulation has to be generated in an XML file;

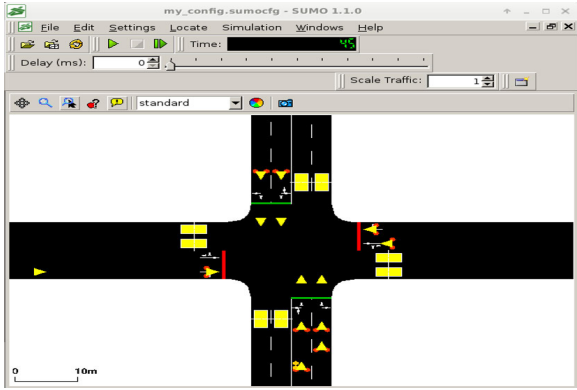


FIGURE 2. Image of the simulated intersection.

each vehicle must have a unique ID, departure time, a route to take, and a vehicle type. Additional parameters can be given, but these are the only ones required. In our simulations, the vehicles' arrival time are uniformly distributed and are generated using a c++ program. An additional parameter known as the relative traffic density (defined later) is used in order to have different levels of traffic flows in different directions. It is introduced to allow the simulations to be as close to the real-world scenarios as possible.

The last part of note in the SUMO toolkit is the Traffic Control Interface (TraCI). TraCI allows the user to directly interact with the simulation and actively retrieve information from simulated objects. This interface is controlled from a python script. The python script used implements the algorithm described in Algorithm 1. The main program takes three arguments: "should the program run with a GUI", the duration of the yellow light phase in seconds, and the number of steps the simulation should simulate. Once started, the main program sets the initial parameters (mostly 2d arrays to store data for vehicles on each road), makes the call to run SUMO, and then steps into a for loop where it simulates a step and then runs the functions listed previously.

B. ESTIMATION OF WAIT TIME T_i

In our simulation, we need knowledge of the vehicle dynamics at the intersection, using which the deadline for any vehicle that enters the queues can be calculated. Specifically, we need to know how long it would take for a car waiting for the green light to pass the intersection after the light changes from red to green. As seen in Lemma 1, the theoretical T_i value relies on several factors that we simply do not have in the simulation. To solve this problem, our strategy is to learn T_i from the SUMO simulation. Combined with the queue lengths information, the T_i value serves two purposes: (i) To determine the deadline of each car and (ii) To decide when to switch traffic light.

Our first step is to track the vehicles near the intersection. This is done by placing induction loops in the simulation. These loops are marked by yellow rectangles in Fig. 2. Each

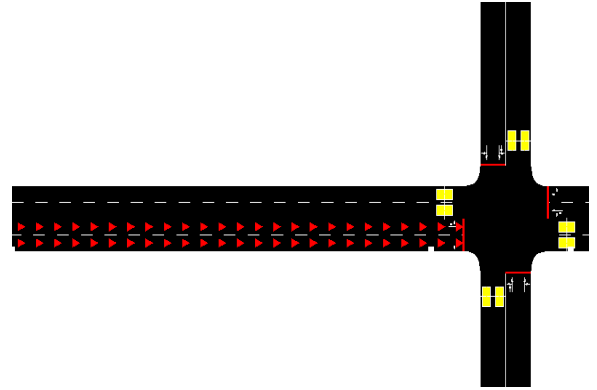


FIGURE 3. Image of the simulated intersection used to measure queue times.

road has one loop 50 meters before the intersection and a second one placed right after the intersection. By doing so, a vehicle is detected when entering and leaving the system, and its system time can be easily figured out.

To gather data for estimated wait times, a special simple simulation as shown in Fig. 3 is run. This simulation begins with 50 vehicles of the same type, and they are all queued with east being the destination direction. The moment the simulation begins, the light in the west/east direction changes from red to green, and the vehicles begin to move through the intersection until they all exit the simulation. The wait times of all 50 vehicles are measured and recorded, and the results are plotted as red dots in Fig. 4 where the horizontal and vertical axes are the queue length and the wait time, respectively. Since these values show a highly linear correlation, a linear regression function `LinearRegression()` from numpy is used to generate a formula to estimate the queue time.

Let $\tilde{T}_i$ be the estimated wait time in our simulation. The linear regression indicates that:

$$\tilde{T}_i = 0.65064 \times i + 1.89094 \quad (25)$$

It is worth noting that time is faster in the simulation than in the real-world. That is why the difference of wait time between adjacent vehicles in the queue is merely 0.65064s. Finally, with the help of (25), the controller can assign deadlines to vehicles based on (23) and (24).

C. PERFORMANCE COMPARISON

We now present the simulation results of ATC-SD and ATC-DD, compared with three other controllers:

The default controller: this is a simple fixed-cycle controller.

The actuated controller: it uses a gap-based actuated control. In particular, it prolongs traffic phases when detecting a continuous stream of traffic and switches to the next phase after a large time gap is detected between successive vehicles.

The delayed based controller [38]: it is a modified form of the actuated control that keeps track of the vehicles delay

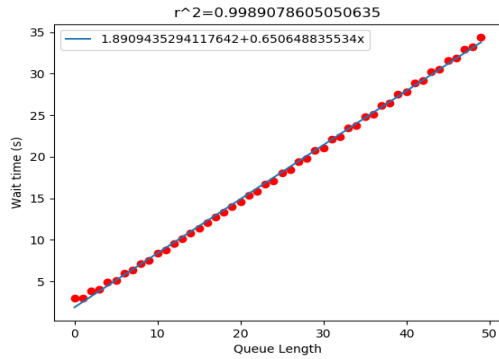


FIGURE 4. Plot of the travel times and the linear model.

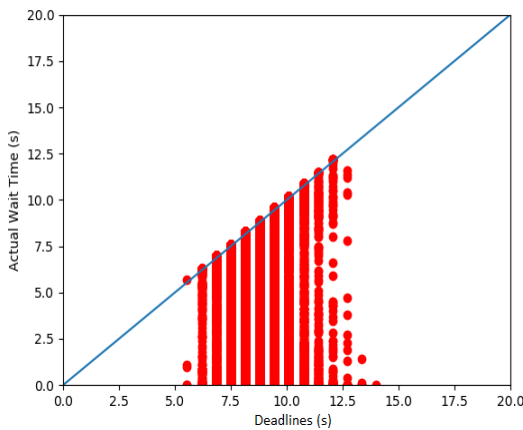


FIGURE 5. Plot of deadline vs actual wait time.

times and changes the light phase when a vehicle approaches a critical delay time.

All three controllers come with SUMO, and we used the default parameters provided by SUMO. More details of these controllers and their parameter settings can be found in SUMO's traffic light documentation [39].

We first consider a simple case that all directions have the same number of vehicles, and the traffic is light. In particular, we have multiple runs, and there are 30,000 vehicles with random arrival times in each run. The ATC-SD controller is tested, and a static deadline is assigned to each vehicle upon its arrival. It is consistent in all the runs that every vehicle is able to leave the intersection by the given deadline. Fig. 5 plots the deadlines of all cars vs. the actual wait times in one of the runs.

After we verify in simulation that the vehicles' assigned deadlines can be guaranteed, we compare the performance of ATC-SD with the other three controllers. Fig. 6 shows a graph containing the various vehicle wait times separated into bins with increments of 5s. This graph clearly shows that the two highest performs in this test are the delayed based traffic light controller and ATC-SD. In this run, as well as other test runs, ATC-SD has a higher number of vehicles that spend less than 1 second than the delay based approach. This amount is significant enough that there are fewer vehicles in all other bins for ATC-SD than the delay based controller. In Table 1,

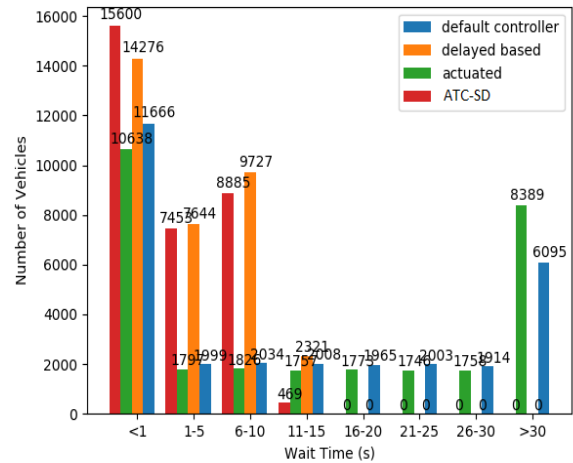


FIGURE 6. Graph of wait times with different traffic light controllers.

TABLE 1. Vehicle wait time comparison.

Controller	Average	Standard deviation	Worst
Default Controller	14.08s	15.34s	47s
Delay Based	3.41s	3.90s	11.3s
Actuated	17.52s	18.11s	55s
ATC-SD	2.85s	3.47s	13.5s

we compare the performance of various controllers in terms of vehicle wait time. Specifically, ATC-SD has the lowest average wait time and standard deviation. Its worst-case wait time is the second best, trailing the delay-based approach by 2.2s.

The performance of traffic light controllers needs be tested in various scenarios. In order to do so, we define a new parameter known as *relative traffic density* and use ρ to represent it:

$$\rho = \frac{\text{\# of cars in the north and south bound}}{\text{\# of cars in the west and east bound}} \times 100\%$$

In what follows, the simulation is set up so that the number of vehicles in the west and east bound directions is used as the base line to determine the traffic density in the north and south bound directions. Specifically, we let ρ be within 0% and 100%: when ρ is near 0%, we have the west and east bound traffic much heavier than the north and south bound traffic; when ρ is near 100%, we have roughly the same level of traffic in all directions. In the rest of the simulation, we let ρ vary between 20% and 100% in steps of 20%. On top of the relative traffic density, we also modify the *absolute traffic density* by varying the inter-arrival time of vehicles. In particular, we have 6 different inter-arrival times, ranging from 0.5s to 1s in steps of 0.1s.

Using the settings described above, we run simulation on four different traffic light controllers: the default one, the delay based controller, ATC-SD, and ATC-DD. Fig. 7 shows the average wait time results of all the controllers for six different inter-arrival times. The horizontal axes are the ρ value. In each subfigure, the left side shows the results when the relative traffic density is small, and as you move to the

Average Wait Time Result

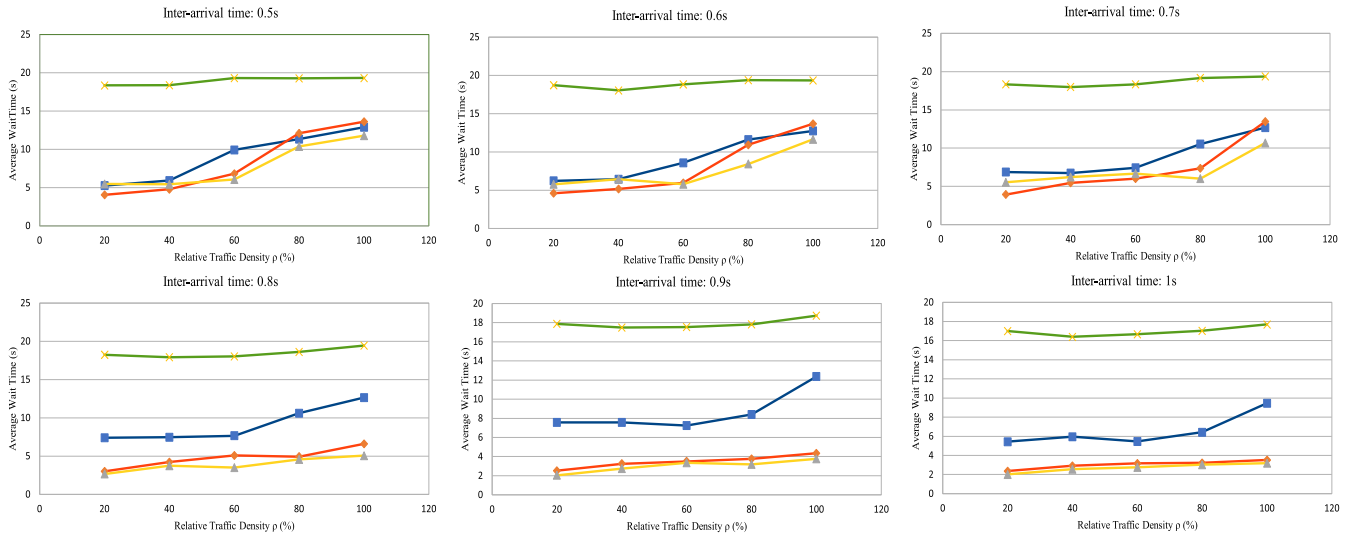


FIGURE 7. Average wait time results of four controllers: the default one is represented by the “x” line; the delay-based one is represented by the square line; ATC-SD is represented by the diamond line; and ATC-DD is represented by the Δ line.

Worst-Case Wait Time Result

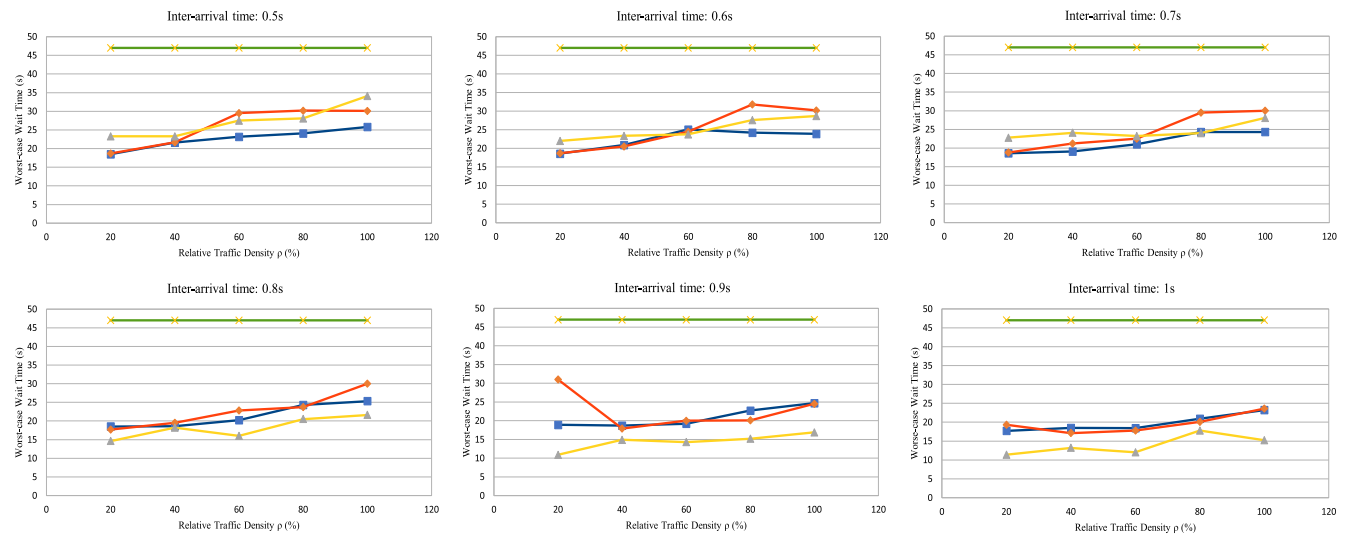


FIGURE 8. Worst-case wait time results of four controllers: the default one is represented by the “x” line; the delay-based is represented by the square line; ATC-SD is represented by the diamond line; and ATC-DD is represented by the Δ line.

right, the relative traffic density increases, until it hits 100%. When the inter-arrival time is below 0.8s, i.e., the traffic is heavy, all the controllers except for the default one are fairly competitive, regardless of the value of ρ . One observation is that the delay based controller occasionally does not do as well as ATC-SD and ATC-DD. The results also indicate that ATC-DD's performance is better than that of ATC-SD when ρ values are large, and it is clearly the winner of all scenarios when ρ is 80% of greater. When the inter-arrival time is equal to or greater than 0.8s, i.e., the traffic is light, ATC-SD and ATC-DD outperform the other two controllers by a large percentage for all ρ values. Between ATC-SD

and ATC-DD, the latter outperforms the former for all ρ values.

The worst-case wait time results are available in Fig. 8. It can be seen that when the inter-arrival time is large (above 0.7s), i.e., the traffic is light, ATC-DD is significantly better than all other controllers. When the inter-arrival time is small (below 0.8s), i.e., the traffic is heavy, the performance depends on the ρ value. Specifically, when ρ is small, the worst-case wait times of the delay-based approach and ATC-SD outperform the default one and ATC-DD; when ρ is large, the delay-based controller is the best. We need to point out that although our controllers are not as good as

the delay-based method in this specific scenario, ATC-DD does perform better in terms of average wait time in the same scenario, as mentioned above. Our simulation results also verify that both ATC-SD and ATC-DD do not suffer from instability issues that toggle the traffic signal back and forth.

VII. CONCLUSION AND FUTURE WORK

In this paper, we model the process of cars passing through an isolated intersection and come up with the shortest and the longest time they could possibly spend at the intersection. The results are significant because they help people better understand how traffic light controllers can be designed in order to provide quality-of-service (QoS) to travelers in intelligent transportation systems. Using these information, we also develop two novel adaptive controllers: ATC-SD and ATC-DD, which assign deadlines to vehicles based on the system state. We use SUMO to simulate the controllers and evaluate their performance. Our results indicate that in our testing scenarios, the assigned vehicles deadlines can be guaranteed. The simulations also indicate that our methods are way better than the fixed-cycle scheduling in all scenarios. Compared with other traffic controllers, our approaches incur smaller average wait time in all scenarios and smaller worst-case wait time in some cases.

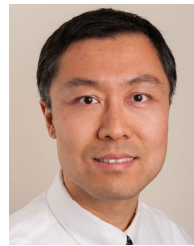
One direction of future work is to verify the performance of ATC-SD and ATC-DD for a system with more than 2 phases. Note that we do not specifically determine the sequence of phases in this work; instead, the sequence of phases is automatically taken care of: in particular, the queue that has a car whose deadline is about to be missed will be given the green time. Therefore, we believe that the proposed adaptive traffic controllers are also suitable for a system with more than 2 phases.

Although our theoretical analysis is mostly applicable to CAVs, the adaptive traffic light controllers we propose in this paper could be used for other types of vehicles. In the future, we are particularly interested in studying how the social aspects and human factors affect the QoS provisioning in adaptive traffic light control when human-driven vehicles and CAVs share the same lane. Our future work also involves improving the adaptive controllers using predicted future information and machine learning, especially for a network of traffic intersections.

REFERENCES

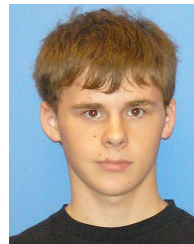
- [1] L. Miao and L. Xu, "Toward intelligent traffic light control with quality-of-service provisioning," in *Proc. ITITS*, 2017, pp. 290–298.
- [2] C. G. Panayiotou, W. C. Howell, and M. Fu, "Online traffic light control through gradient estimation using stochastic fluid models," *IFAC Proc. Volumes*, vol. 38, no. 1, pp. 90–95, 2005.
- [3] Y. Geng and C. G. Cassandras, "Traffic light control using infinitesimal perturbation analysis," in *Proc. IEEE 51st IEEE Conf. Decis. Control (CDC)*, Dec. 2012, pp. 7001–7006.
- [4] Y. Geng and C. G. Cassandras, "Quasi-dynamic traffic light control for a single intersection," in *Proc. IEEE 52nd IEEE Conf. Decis. Control*, Dec. 2013, pp. 880–885.
- [5] Y. Geng and C. G. Cassandras, "Multi-intersection two-way traffic light control with blocking using infinitesimal perturbation analysis," in *Proc. IEEE Int. Conf. Control Appl. (CCA)*, Aug. 2013, pp. 382–387.
- [6] A. DiFebraro, D. Giglio, and N. Sacco, "Urban traffic control structure based on hybrid Petri nets," *IEEE Trans. Intell. Transp. Syst.*, vol. 5, no. 4, pp. 224–237, Dec. 2004.
- [7] Y.-S. Huang, Y.-S. Weng, and M. Zhou, "Modular design of urban traffic-light control systems based on synchronized timed Petri nets," *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 2, pp. 530–539, Apr. 2014.
- [8] G. F. List and M. Mashayekhi, "A modular colored stochastic Petri net for modeling and analysis of signalized intersections," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 3, pp. 701–713, Mar. 2016.
- [9] L. Qi, M. Zhou, and W. Luan, "Emergency traffic-light control system design for intersections subject to accidents," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 1, pp. 170–183, Jan. 2016.
- [10] Y.-S. Huang, Y.-S. Weng, and M. Zhou, "Design of traffic safety control systems for emergency vehicle preemption using timed Petri nets," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 4, pp. 2113–2120, Aug. 2015.
- [11] L. Qi, M. Zhou, and W. Luan, "A two-level traffic light control strategy for preventing incident-based urban traffic congestion," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 1, pp. 13–24, Jan. 2018.
- [12] J. Luo, Y.-S. Huang, and Y.-S. Weng, "Design of variable traffic light control systems for preventing two-way grid network traffic jams using timed Petri nets," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 7, pp. 3117–3127, Jul. 2020.
- [13] C. P. Pappis and E. H. Mamdani, "A fuzzy logic controller for a traffic junction," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-7, no. 10, pp. 707–717, Oct. 1977.
- [14] Y. S. Murat and E. Gedizlioglu, "A fuzzy logic multi-phased signal control model for isolated junctions," *Transp. Res. C, Emerg. Technol.*, vol. 13, no. 1, pp. 19–36, Feb. 2005.
- [15] M. Collotta, L. L. Bello, and G. Pau, "A novel approach for dynamic traffic lights management based on wireless sensor networks and multiple fuzzy logic controllers," *Expert Syst. Appl.*, vol. 42, no. 13, pp. 5403–5415, Aug. 2015.
- [16] C. Li and S. Shimamoto, "An open traffic light control model for reducing vehicles' CO₂ emissions based on ETC vehicles," *IEEE Trans. Veh. Technol.*, vol. 61, no. 1, pp. 97–110, Jan. 2012.
- [17] A. Ahmad, R. Arshad, H. S. Al-Raweshidy, S. A. Mahmud, and G. M. Khan, "Earliest-deadline-based scheduling to reduce urban traffic congestion," *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 4, pp. 1510–1526, Aug. 2014.
- [18] F. Ahmad, S. A. Mahmud, and F. Z. Yousaf, "Shortest processing time scheduling to reduce traffic congestion in dense urban areas," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 47, no. 5, pp. 838–855, May 2017.
- [19] Z. Cao, S. Jiang, J. Zhang, and H. Guo, "A unified framework for vehicle rerouting and traffic light control to reduce traffic congestion," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 7, pp. 1958–1973, Jul. 2017.
- [20] A. A. Zaidi, B. Kulcsar, and H. Wymeersch, "Back-pressure traffic signal control with fixed and adaptive routing for urban vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 8, pp. 2134–2143, Aug. 2016.
- [21] L. A. Prashanth and S. Bhatnagar, "Reinforcement learning with average cost for adaptive control of traffic lights at intersections," in *Proc. 14th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2011, pp. 1640–1645.
- [22] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, "Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (MARLIN-ATSC): Methodology and large-scale application on downtown Toronto," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1140–1150, Sep. 2013.
- [23] M. Zhou, Y. Yu, and X. Qu, "Development of an efficient driving strategy for connected and automated vehicles at signalized intersections: A reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 433–443, Jan. 2020.
- [24] M. Stevens and C. Yeh, "Reinforcement learning for traffic optimization," Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, Tech. Rep., 2016. [Online]. Available: <http://cs229.stanford.edu/proj2016spr047.pdf>
- [25] E. Björck and F. Omstedt, "A comparison of algorithms used in traffic control systems," Degree Project Technol., First Cycle, 15 Credits, School Elect. Eng. Comput. Sci., KTH Roy. Inst. Technol., Stockholm, Sweden, Tech. Rep., 2018.
- [26] L. A. Prashanth and S. Bhatnagar, "Threshold tuning using stochastic optimization for graded signal control," *IEEE Trans. Veh. Technol.*, vol. 61, no. 9, pp. 3865–3880, Nov. 2012.
- [27] H. Ge, Y. Song, C. Wu, J. Ren, and G. Tan, "Cooperative deep Q-learning with Q-value transfer for multi-intersection signal control," *IEEE Access*, vol. 7, pp. 40797–40809, 2019.

- [28] Y. Dujardin, F. Boillot, D. Vanderpooten, and P. Vinant, "Multiobjective and multimodal adaptive traffic light control on single junctions," in *Proc. 14th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2011, pp. 1361–1368.
- [29] Y. Zhang, K. Gao, Y. Zhang, and R. Su, "Traffic light scheduling for pedestrian-vehicle mixed-flow networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 4, pp. 1468–1483, Apr. 2019.
- [30] K. Gao, Y. Zhang, Y. Zhang, R. Su, and P. N. Suganthan, "Meta-heuristics for bi-objective urban traffic light scheduling problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 7, pp. 2618–2629, Jul. 2019.
- [31] M. A. Khamis, W. Gomaa, A. El-Mahdy, and A. Shoukry, "Adaptive traffic control system based on Bayesian probability interpretation," in *Proc. Jpn.-Egypt Conf. Electron., Commun. Comput.*, Mar. 2012, pp. 151–156.
- [32] V. Protschky, K. Wiesner, and S. Feit, "Adaptive traffic light prediction via Kalman filtering," in *Proc. IEEE Intell. Vehicles Symp.*, Jun. 2014, pp. 151–157.
- [33] L. Tong, Z. Guan, M. He, and L. Xiong, "Signal timing model of urban intersection based on double restraint gravitational model," *J. Eng.*, vol. 2019, no. 23, pp. 9162–9165, Dec. 2019.
- [34] D. Rahbari, "Help the genetic algorithm to minimize the urban traffic on intersections," *Int. J. Res. Comput. Sci.*, vol. 4, no. 4, pp. 1–9, 2014.
- [35] P. Jiao, H. Wang, and T. Sun, "Real-time arterial coordination control based on dynamic intersection turning fractions estimation using genetic algorithm," *Math. Problems Eng.*, vol. 2014, Jul. 2014, Art. no. 430497.
- [36] E. M. Ahmed, "Continuous genetic algorithm for traffic signal control," in *Proc. Int. Conf. Comput., Control, Electr., Electron. Eng. (ICCCEEE)*, Aug. 2018, pp. 1–5.
- [37] L. Xu, L. Y. Wang, G. Yin, and H. Zhang, "Communication information structures and contents for enhanced safety of highway vehicle platoons," *IEEE Trans. Veh. Technol.*, vol. 63, no. 9, pp. 4206–4220, Nov. 2014.
- [38] R. Oertel and P. Wagner, "Delay-time actuated traffic signal control for an isolated intersection," in *Proc. 90th Annu. Meeting Transp. Res. Board (TRB)*, 2011, pp. 1–13.
- [39] *Sumo Traffic Lights Documentation*. Accessed: Feb. 10, 2021. [Online]. Available: https://sumo.dlr.de/docs/Simulation/Traffic_Lights.html



LEI MIAO received the B.S. and M.S. degrees from Northeastern University, Shenyang, Liaoning, China, in 1998 and 2001, respectively, and the Ph.D. degree from Boston University, Boston, MA, USA, in 2006.

From 2006 to 2009, he was with Nortel Networks, Billerica, MA, USA, where he conducted research and development for the metro ethernet routing switches 8600. From 2009 to 2011, he was a Visiting Professor with the School of Electronics and Computing Systems, University of Cincinnati. From 2011 to 2014, he was with NuVo Technologies/Legrand North America, Hebron, KY, USA, where he performed research and development for the NuVo wireless audio players. From 2014 to 2015, he was a Faculty Member with the Department of Electrical and Computer Engineering Technology, State University of New York, Farmingdale. He is currently an Associate Professor of Mechatronics Engineering with Middle Tennessee State University. His research interests include real-time systems and control and optimization for discrete-event systems with applications to communication networks, wireless networks, and cyber-physical systems. He has published over 30 refereed journal articles and conference papers in these areas.



DALLAS LEITNER received the B.S. and M.S. degrees in engineering technology from Middle Tennessee State University (MTSU), Murfreesboro, TN, USA, in 2015 and 2017, respectively, where he is currently pursuing the Ph.D. degree in computational science.

His research interests include performance evaluation, optimizations, and computer simulation for intelligent transportation systems.

• • •